目录

我用 OpenClaw + Vercel + Supabase 建立了一家 AI 公司 —— 两周后，它们开始自主运行 2

起点：你有了 OpenClaw。然后呢? 2

闭环是什么样的 3

陷阱 1：两个地方在争抢工作 4

陷阱 2：触发了但没人接手 5

陷阱 3：配额满时队列持续增长 7

赋予生命：触发器 + 反应矩阵 8

完整架构 10

你可以带走的经验 11

最后的想法 13

完整教程：运行公司的 6 个 AI Agent —— 我是如何从零开始构建它们的 14

第一章：基础 —— 闭环的 4 张表 15

第二章：让他们开口说话 —— 圆桌会议对话系统 25

每日日程 29

第三章：让他们记住 —— 记忆与学习 32

第四章：赋予他们关系 —— 动态亲密度 37

第五章：让他们提出想法 —— 主动性系统（The Initiative System） 41

第六章：赋予它们个性——声音进化（Voice Evolution） 43

第七章：让它看起来很酷——前端 46

第八章：发布清单 49

最后的一点想法 57

我用 OpenClaw + Vercel + Supabase 建立了一家 AI 公司 —— 两周后，它们开始自主运行

Vox @Voxyz_ai

https://x.com/Voxyz_ai/status/2019914775061270747

https://www.voxyz.space/, https://github.com/Heyvhuang/ship-faster

6 个 AI agent，1 台 VPS，1 个 Supabase 数据库 —— 从“agent 会聊天”到“agent 自主运行网站”花了我两周时间。这篇文章将详细介绍这中间缺失的关键环节、如何修复它们，以及一套你可以直接拿走使用的架构。

起点：你有了 OpenClaw。然后呢?

如果你最近一直在玩 AI agent，很可能你已经配置好了 OpenClaw。

它解决了一个大问题：让 Claude 能够使用工具、浏览网页、操作文件并运行定时任务。你可以给 agent 分配 cron 任务 —— 每日推文、每小时情报扫描、定期研究报告。

我也是从这里开始的。

我的项目叫做 VoxYZ Agent World —— 6 个 AI agent 在一个像素艺术风格的办公室内自主运营一个网站。技术栈很简单：

- **OpenClaw**（运行在 VPS 上）：agent 的“大脑” —— 运行圆桌讨论、cron 任务、深度研究。
- **Next.js + Vercel**：网站前端 + API 层。
- **Supabase**：所有状态（提案、任务、事件、记忆）的单一事实来源。

六个角色，各司其职：Minion 负责决策，Sage 分析策略，Scout 收集情报，Quill 撰写内容，Xalt 管理社交媒体，Observer 进行质量检查。

OpenClaw 的 cron 任务让他们每天“准时上班”。圆桌会议让他们讨论、投票并达成共识。

但这只是“会聊天”，而不是“会运营”。

agent 产出的所有内容 —— 推文草案、分析报告、内容文章 —— 都停留在 OpenClaw 的输出层。没有任何机制将其转化为实际执行，也没有任何机制在执行完成后告诉系统“已完成”。

在“agent 能产出内容”和“agent 能端到端运行”之间，缺少一个完整的 执行 → 反馈 → 重新触发 的闭环。这就是本文要讨论的内容。

闭环是什么样的

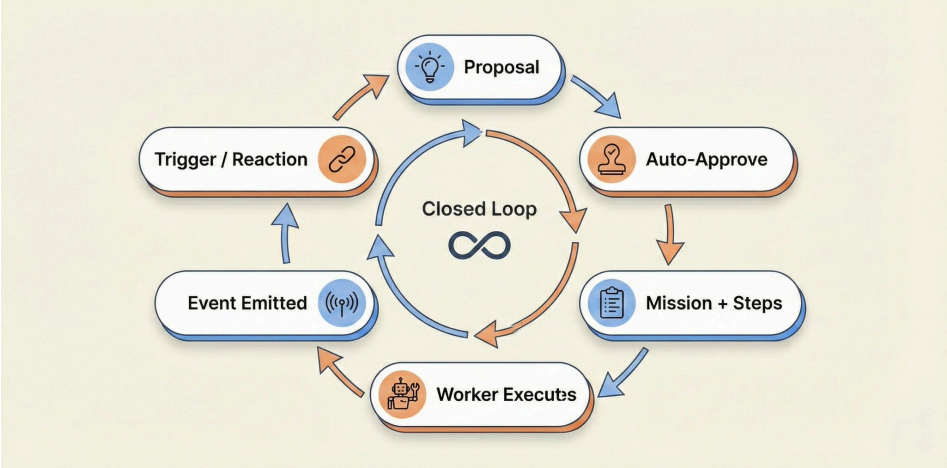
让我们先定义一下“闭环”，以免构建出错误的东西。

一个真正无需人工干预的 agent 系统需要运行这个循环：

```
Agent 提出想法（提案 Proposal）
  ↓
```

自动审批检查（自动审批 Auto-Approve）
↓
创建任务 + 步骤（任务 Mission + 步骤 Steps）
↓
Worker 领取并执行（Worker）
↓
发出事件（事件 Event）
↓
触发新的反应（触发 Trigger / 反应 Reaction）
↓
回到第一步

听起来很简单？在实践中，我遇到了三个陷阱 —— 每一个都让系统“看起来在运行，但实际上在原地打转”。



陷阱 1：两个地方在争抢工作

我的 VPS 上有 OpenClaw worker 在领取并执行任务。与此同时，Vercel 有一个心跳 cron 运行任务 worker（mission-worker），也试图领取同样的任务。

双方都在查询同一张表，抓取同一个步骤，独立执行。没有协调，纯粹的竞态条件。偶尔一个步骤会被双方标记上冲突的状态。

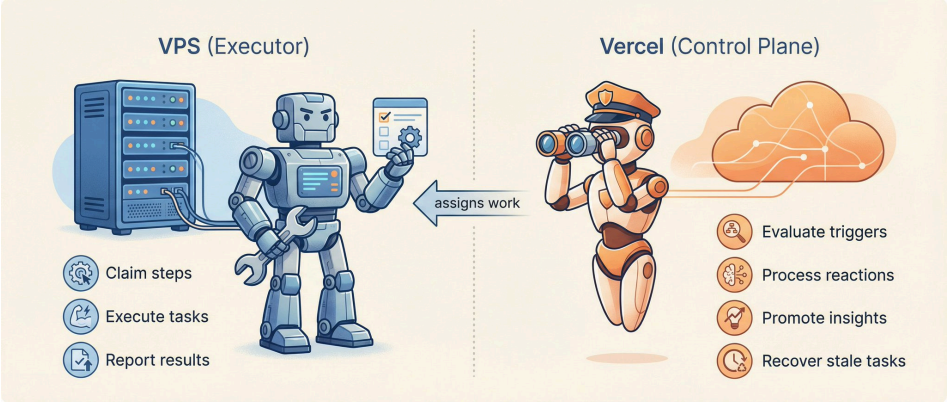
修复：砍掉一个。VPS 是唯一的执行者。Vercel 只运行轻量级控制平面（评估触发器、处理反应队列、清理卡住的任务）。

改动非常小 —— 从 heartbeat 路由中移除 runMissionWorker 调用：

```
// Heartbeat 现在只做 4 件事
const triggerResult = await evaluateTriggers(sb, 4_000);
const reactionResult = await processReactionQueue(sb, 3_000);
const learningResult = await promoteInsights(sb);
const staleResult = await recoverStaleSteps(sb);
```

额外奖励：节省了 Vercel Pro 的成本。Heartbeat 不再需要 Vercel 的 cron —— VPS 上的一行 crontab 就能搞定：

/5 * * * curl -s -H “Authorization: Bearer $KEY” https://yoursite.com/api/ops/heartbeat



陷阱 2：触发了但没人接手

我写了 4 个触发器：推文爆火时自动分析、任务失败时自动诊断、内容发布时自动审核、见解成熟时自动晋升。

在测试期间我注意到：触发器正确检测到了条件并创建了提案（proposal）。但提案一直处于 pending（待处理）状态 —— 从未变成任务（mission），也从未生成可执行步骤。

原因：触发器直接向 ops_mission_proposals 表插入数据，但正常的审批工作流是：插入提案 → 评估自动审批 → 如果获批，创建任务 + 步骤。触发器跳过了最后两步。

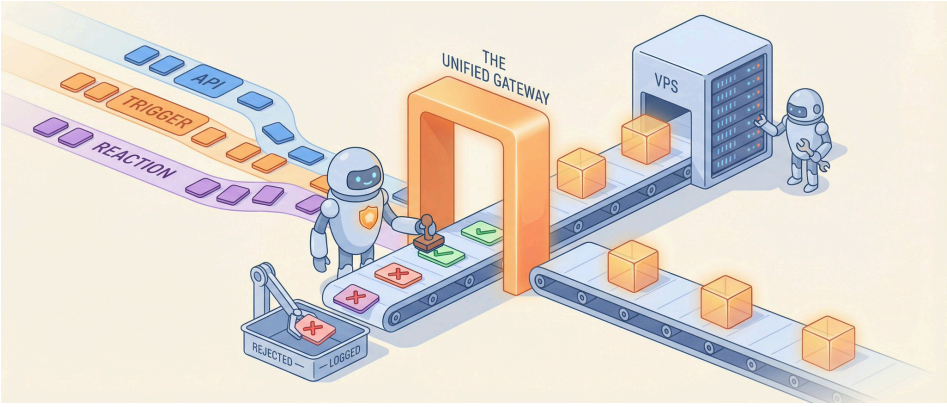
修复：提取一个共享函数 createProposalAndMaybeAutoApprove。每个创建提案的路径 —— API、触发器、反应 —— 都必须调用这一个函数。

```
// proposal-service.ts — 所有提案创建的唯一入口点
export async function createProposalAndMaybeAutoApprove(
  sb: SupabaseClient,
  input: ProposalServiceInput,  // 包括 source: 'api' | 'trigger' | 'reaction'
): Promise<ProposalServiceResult> {
  // 1. 检查每日限制
  // 2. 检查容量门控（Cap Gates，下文详述）
  // 3. 插入提案
  // 4. 发出事件
  // 5. 评估自动审批
  // 6. 如果获批 → 创建任务 + 步骤
  // 7. 返回结果
}
```

改动后，触发器只需返回一个提案模板。评估器调用该服务：

```
// trigger-evaluator.ts
if (outcome.fired && outcome.proposal) {
  await createProposalAndMaybeAutoApprove(sb, {
    ...outcome.proposal,
    source: 'trigger',
  });
}
```

一个函数统领全局。任何未来的检查逻辑（速率限制、黑名单、新的容量限制）—— 只需修改一个文件。



陷阱 3：配额满时队列持续增长

最隐蔽的 bug —— 表面上看一切正常，日志里没有错误，但数据库中堆积的排队步骤越来越多。

原因：推文配额已满，但提案仍在被批准，生成任务，生成排队步骤。VPS worker 看到配额已满就直接跳过了 —— 既没有领取任务，也没有标记为失败。第二天，又来了一批。

修复：容量门控（Cap Gates）—— 在提案入口处拒绝。从一开始就不让它生成排队步骤。

```
// proposal-service.ts 内部的门控系统
const STEP**KIND**GATES: Record<string, StepKindGate> = {
  write_content: checkWriteContentGate,  // 检查每日内容上限
  post_tweet:    checkPostTweetGate,     // 检查推文配额
  deploy:        checkDeployGate,        // 检查部署策略
};
```

每种步骤类型都有自己的门控。推文配额满了？提案会立即被拒绝，清晰地说明原因，并发出警告事件。没有排队的步骤 = 没有堆积。

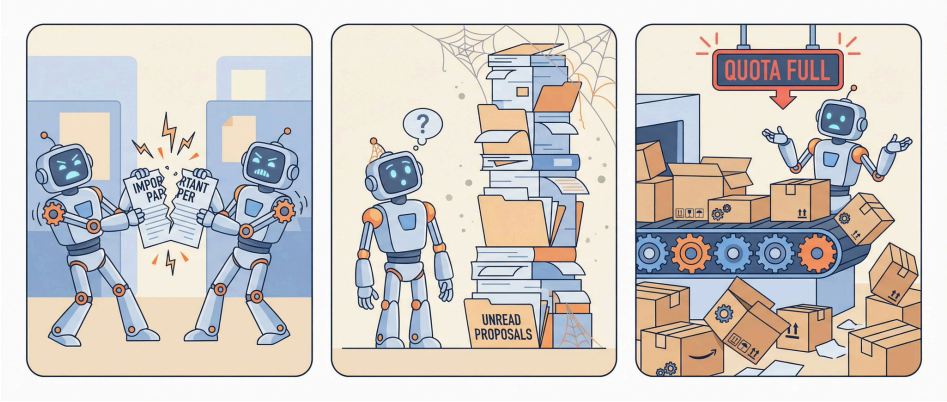
这是 post_tweet 门控：

```
async function checkPostTweetGate(sb: SupabaseClient) {
  const autopost = await getOpsPolicyJson(sb, 'x_autopost', {});
  if (autopost.enabled === false) return { ok: false, reason: 'x_autopost
disabled' };

  const quota = await getOpsPolicyJson(sb, 'x_daily_quota', {});
  const limit = Number(quota.limit ?? 10);
  const { count } = await sb
    .from('ops_tweet_drafts')
    .select('id', { count: 'exact', head: true })
    .eq('status', 'posted')
    .gte('posted_at', startOfTodayUtcIso());

  if ((count ?? 0) >= limit) return { ok: false, reason: `Daily tweet quota
reached (${count}/${limit})` };
  return { ok: true };
}
```

核心原则：在门控处拒绝，不要在队列中堆积。被拒绝的提案会被记录（用于审计），而不是被默默丢弃。



赋予生命：触发器 + 反应矩阵

修复了三个陷阱后，循环可以工作了。但系统目前只是一个“无错误的流水线”，而不是一个“响应迅速的团队”。

触发器

4 个内置规则 —— 每个规则检测一种情况并返回一个提案模板：

条件	动作	冷却时间
推文互动率 > 5%	Growth 分析为何走红	2 小时
任务失败	Sage 诊断根本原因	1 小时
新内容发布	Observer 审查质量	2 小时
洞察获得多次点赞	自动提升至永久记忆	4 小时

触发器只负责检测 —— 它们不直接操作数据库，而是将提案模板交给提案服务。所有的配额门控和自动批准逻辑都会自动应用。

冷却时间很重要。如果没有它，一条走红的推文会在每个心跳周期（每 5 分钟）触发一次分析。

反应矩阵

最有趣的部分 —— agent 之间自发的互动。

存储在 ops_policy 表中的 reaction_matrix：

```
{
 "patterns": [
  { "source": "twitter-alt", "tags": ["tweet","posted"], "target": "growth",
    "type": "analyze", "probability": 0.3, "cooldown": 120 },
  { "source": "*", "tags": ["mission:failed"], "target": "brain",
    "type": "diagnose", "probability": 1.0, "cooldown": 60 }
 ]
}
```

Xalt 发布了一条推文 → 30% 的概率 Growth 会分析其表现。任何任务失败 → 100% 的概率 Sage 会进行诊断。

probability（概率）不是 bug，而是一个特性。100% 的确定性 = 机器人。加入随机性 = 感觉更像一个真实的团队，“有时有人回应，有时则没有”。

自愈：系统总会卡住

VPS 重启、网络波动、API 超时 —— 步骤会卡在 running 状态，而实际上没有人正在处理它们。

心跳机制包含 recoverStaleSteps：

```
// 30 分钟无进展 → 标记为失败 → 检查任务是否应完成
const STALE**THRESHOLD**MS = 30 * 60 * 1000;

const { data: stale } = await sb
  .from('ops_mission_steps')
  .select('id, mission_id')
  .eq('status', 'running')
  .lt('reserved_at', staleThreshold);

for (const step of stale) {
  await sb.from('ops_mission_steps').update({
    status: 'failed',
    last_error: 'Stale: no progress for 30 minutes',
  }).eq('id', step.id);
  await maybeFinalizeMissionIfDone(sb, step.mission_id);
}
```

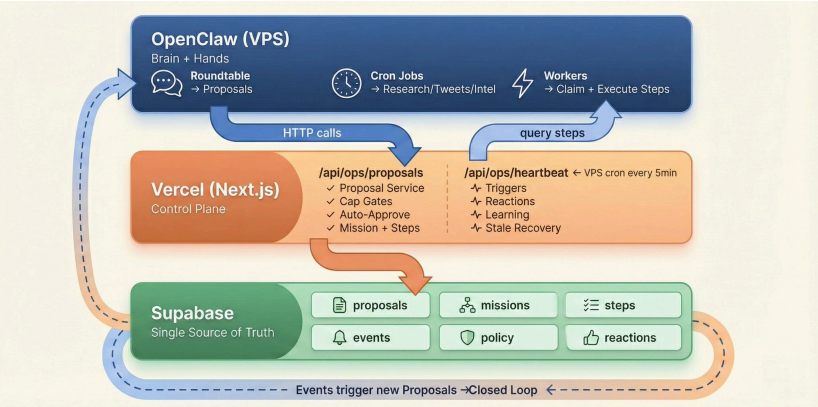
maybeFinalizeMissionIfDone 会检查任务中的所有步骤 —— 任何一个步骤失败都意味着整个任务失败，所有步骤完成则意味着成功。不再有“一个步骤成功就将整个任务标记为成功”的情况。

完整架构

具有清晰职责的三个层级：

- **OpenClaw (VPS)**：思考 + 执行（大脑 + 双手）
- **Vercel**：批准 + 监控（控制平面）

- **Supabase**：所有状态（共享皮层）



你可以带走的经验

如果你拥有 OpenClaw + Vercel + Supabase，这里有一个最小可行闭环清单：

1. 数据库表 (Supabase)

你至少需要这些：

表名	用途
ops_mission_proposals	存储提案（待定/已接受/已拒绝）
ops_missions	存储任务（已批准/运行中/已成功/已失败）
ops_mission_steps	存储执行步骤（排队中/运行中/已成功/已失败）
ops_agent_events	存储事件流（所有 agent 动作）
ops_policy	存储策略（auto_approve、x_daily_quota 等，以 JSON 格式存储）
ops_trigger_rules	存储触发规则
ops_agent_reactions	存储反应队列
ops_action_runs	存储执行日志

2. 提案服务（单个文件）

将提案创建 + 配额限制 + 自动批准 + 任务创建放在一个函数中。所有来源（API、触发器、反应）都调用它。这是整个循环的枢纽。

3. 策略驱动的配置 (ops_policy 表)

不要硬编码限制。每个行为开关都存在于 ops_policy 表中：

```
// auto_approve: 哪些步骤类型允许自动通过
{ "enabled": true, "allowed_step_kinds":
["draft_tweet","crawl","analyze","write_content"] }

// x_daily_quota: 每日推文上限
{ "limit": 8 }

// worker_policy: Vercel 是否执行步骤（设置为 false = 仅 VPS 执行）
{ "enabled": false }
```

无需重新部署代码即可随时调整策略。

4. 心跳（一个 API 路由 + 一行 Crontab）

Vercel 上的一个 /api/ops/heartbeat 路由。VPS 上的一行 crontab 每 5 分钟调用一次。其内部运行：触发器评估、反应队列处理、洞察转化、过期任务清理。

5. VPS Worker 契约

每种步骤类型都映射到一个 worker。完成一个步骤后，worker 调用 maybeFinalizeMissionIfDone 来检查整个任务是否应该完成。永远不要仅仅因为一个步骤完成就将任务标记为成功。

两周时间线

阶段	时间	完成内容
基础设施	预先存在	OpenClaw VPS + Vercel + Supabase（已设置好）
提案 + 批准	3 天	提案 API + 自动批准 + 策略表
执行引擎	2 天	mission-worker + 8 个步骤执行器
触发器 + 反应	2 天	4 种触发器类型 + 反应矩阵
循环统一	1 天	提案服务 + 配额限制 + 修复三个陷阱
情感系统 + 视觉	2 天	情感系统重写 + 空闲行为 + 像素办公室集成
数据初始化 + 上线	半天	数据库迁移 + 种子策略 + crontab

排除预先存在的基础设施，核心闭环（提议 → 执行 → 反馈 → 重新触发）大约需要一周时间来打通。

最后的想法

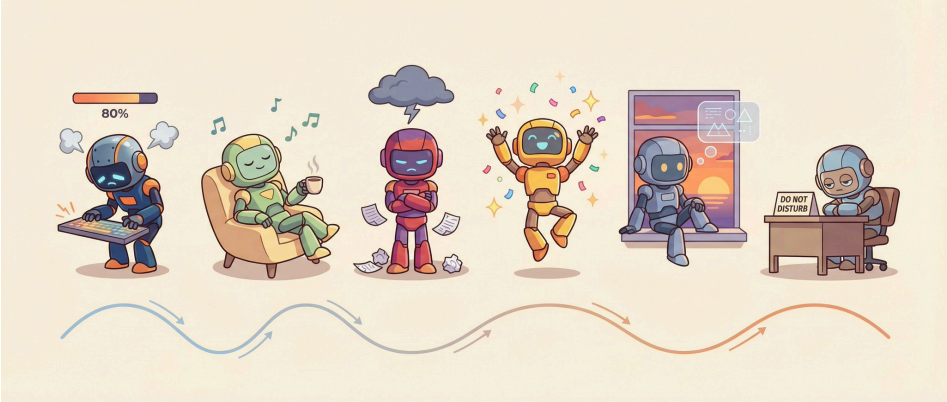
这 6 个 agent 现在每天都在自主运行 voxyz.space。我仍然每天在优化系统 —— 调整策略，扩展触发规则，改进 agent 之间的协作方式。

它远非完美 —— agent 间的协作仍然很基础，“自由意志”大多是通过基于概率的非确定性来模拟的。但系统确实在运行，确实不需要有人盯着它。

下一篇文章，我将介绍 agent 如何相互“争论”和“说服” —— 圆桌投票和 Sage 的记忆整合如何将 6 个独立的 Claude 实例转变为类似于团队认知的东西。

如果你正在使用 OpenClaw 构建 agent 系统，我很乐意交流心得。当你作为一个独立开发者做这件事时，每一次对话都能让你少踩一个坑。

完整教程：运行公司的 6 个 AI Agent —— 我是如何从零开始构建它们的



Vox @Voxyz_ai

https://x.com/Voxyz_ai/status/2020272022417289587

我的上一篇文章火了 —— 60 万次浏览，2400 个点赞。最常见的回复是？“我明白了，但我自己做不出来。”所以我写了这份构建指南。5600 字，每一个步骤，毫无保留。你不需要知道如何编写代码 —— 只需要知道如何与 AI 编程助手对话。

你最终将获得什么

当你完成后，你将拥有：

- 6 个每天进行实际工作的 AI Agent：扫描情报、撰写内容、发布推文、运行分析。
- 每天 10-15 场对话：站会、辩论、茶水间闲聊、一对一指导。
- 能够记住经验教训并将其纳入未来决策的 Agent。
- 会发生变化的关系 —— 协作越多，亲密度越高；争论过多，亲密度下降。
- 不断进化的说话风格 —— 一个拥有丰富“推文参与度”经验的 Agent 会开始自然地引用参与策略。
- 全透明化 —— 前端有一个像素风办公室，实时显示一切。技术栈：Next.js + Supabase + VPS。每月成本：8 美元固定费用 + LLM 使用费。

没有 OpenAI Assistants API。没有 LangChain。没有 AutoGPT。只有 PostgreSQL + 几个 Node.js worker + 一个规则引擎。

你不需要从 6 个 Agent 开始。从 3 个开始 —— 一个协调者、一个执行者和一个观察者 —— 你就会拥有一个完整运行的闭环。

第一章：基础 —— 闭环的 4 张表

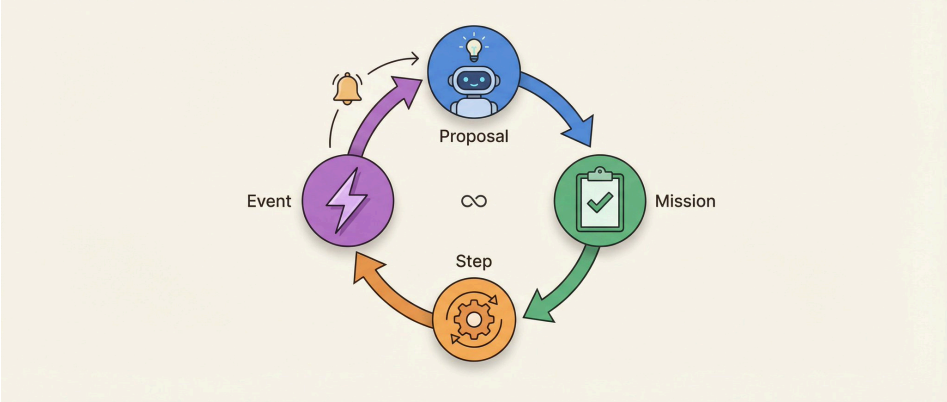
很多人直接跳到“自主思考”。但如果你的 Agent 甚至无法处理队列中的步骤，我们谈论的是哪种自主性？

核心数据模型

整个系统骨架由 4 张表组成。它们之间的关系很简单 —— 想象一个圆圈：

Agent 提出一个想法 (Proposal) → 获得批准并成为一项任务 (Mission) → 分解为具体的步骤 (Step) → 执行触发一个事件 (Event) → 事件触发一个新想法 → 回到第一步。

这就是循环。它永远运行。这就是你的“闭环”。



在 Supabase 中创建这些表：

核心数据模型 —— 4 张表：

📋 ops_mission_proposals

→ 存储提案

→ 字段：agent_id, title, status (pending/accepted/rejected), proposed_steps

📋 ops_missions

→ 存储任务

→ 字段：title, status (approved/running/succeeded/failed), created_by

📋 ops_mission_steps

→ 存储执行步骤

→ 字段：mission_id, kind (draft_tweet/crawl/analyze…), status (queued/running/succeeded/failed)

📋 ops_agent_events

→ 存储事件流

→ 字段：agent_id, kind, title, summary, tags[]

初学者提示：如果你不知道如何编写 SQL，复制上面的表格并将其粘贴到你的 AI 编程助手，并输入“为这些表生成 Supabase SQL 迁移脚本”。它会处理好的。

提案服务：整个系统的枢纽

初学者提示：什么是提案 (Proposal)？它是 Agent 的“请求”。例如，你的社交媒体 Agent 想要发布一条推文，所以它提交了一个提案：“我想发布关于 AI 趋势的推文。”系统会审核它 ——要么批准它（将其转化为可执行的任务），要么拒绝它（并给出理由）。

这是我犯过的最大错误之一——触发器、API 和反应矩阵（reaction matrix）都在独立地创建提案。有些通过了审批，有些则没有。

修复方法：一个统一的提案入口流水线（单一入口点）。无论提案来自哪里——agent 的主动行为、自动触发器，还是另一个 agent 的反应——所有内容都通过同一个函数处理。

```
// proposal-service.ts — 提案创建的单一入口点
export async function createProposalAndMaybeAutoApprove(sb, input) {
  // 1. 检查此 agent 是否达到了每日限制
  // 2. 检查容量门控 Cap Gates（推文配额满了？今天内容太多了?）
  //    → 如果已满，立即拒绝——不创建排队步骤
  // 3. 插入提案
  // 4. 评估自动审批（低风险任务自动通过）
  // 5. 如果获批 → 创建任务（mission）+ 步骤（steps）
  // 6. 触发事件（以便前端可以看到它）
}
```

什么是容量门控（Cap Gates）？你可以这样想：你的公司有一条规则——每天最多发布 8 条推文。如果你不在“提交请求”这一步检查配额，会发生什么？请求仍然会被批准，任务仍然会进入队

列，执行器检查后说“我们今天已经发布了 8 条了”并跳过它——但该任务仍然留在队列中。任务会堆积起来，除非你手动检查数据库，否则你不会注意到。

因此，要在提案入口点进行检查——配额已满意味着立即拒绝，没有任务进入队列。

```
const STEP**KIND**GATES = {
  write_content: checkWriteContentGate,  // 检查每日内容限制
  post_tweet:    checkPostTweetGate,     // 检查推文配额
  deploy:        checkDeployGate,        // 检查部署策略
};
```

每种步骤类型都有自己的门控。推文门控会检查今天已发布的数量与配额的对比：

```
async function checkPostTweetGate(sb) {
  const quota = await getPolicy(sb, 'x_daily_quota');   // 从 ops_policy 表读取
  const todayCount = await countTodayPosted(sb);        // 计算今日已发布数量
  if (todayCount >= quota.limit) {
    return { ok: false, reason: `配额已满 (${todayCount}/${quota.limit})` };
  }
  return { ok: true };
}
```

提示：在入口点进行拦截，不要让任务在队列中堆积。被拒绝的提案应该被记录（用于审计追踪），而不是被无声无息地丢弃。

策略表：ops_policy

不要在代码中硬编码配额和功能开关（feature flags）。将所有内容存储在具有键值结构的 ops_policy 表中：

```
CREATE TABLE ops_policy (
  key TEXT PRIMARY KEY,
  value JSONB NOT NULL DEFAULT '{}',
  updated_at TIMESTAMPTZ DEFAULT now()
);
```

一些核心策略：

```
// auto_approve: 哪些步骤类型可以自动审批
{ "enabled": true, "allowed_step_kinds":
["draft_tweet","crawl","analyze","write_content"] }

// x_daily_quota: 每日推文限制
{ "limit": 8 }

// content_policy: 内容控制
{ "enabled": true, "max_drafts_per_day": 8 }
```

好处是：你可以通过在 Supabase 仪表板中编辑 JSON 值来调整任何策略——无需重新部署。系统在凌晨 3 点失控了？只需将 enabled 切换为 false 即可。

Heartbeat：系统的脉搏

初学者提示：什么是 Heartbeat（心跳）？字面意思——就是心跳。你的心脏每秒跳动一次以保持血液流动；系统的 heartbeat 每 5 分钟触发一次，检查所有需要检查的内容。没有它，提案将无人审查，触发器将无人评估，卡住的任务将无法恢复——系统就会像心电图直线一样失去生机。

每 5 分钟触发一次，执行 6 件事：

```
// /api/ops/heartbeat — Vercel API 路由
export async function GET(req) {
  // 1. 评估触发器（是否有任何条件满足?）
  const triggers = await evaluateTriggers(sb, 4000);
  // 2. 处理反应队列（agent 是否需要交互?）
  const reactions = await processReactionQueue(sb, 3000);
  // 3. 提升洞察（是否有任何值得晋升的发现?）
  const learning = await promoteInsights(sb);
  // 4. 从结果中学习（那些推文表现如何？编写经验教训）
  const outcomes = await learnFromOutcomes(sb);
  // 5. 恢复卡住的任务（运行 30 分钟以上且无进展的步骤 → 标记为失败）
  const stale = await recoverStaleSteps(sb);
```

```
  // 6. 恢复卡住的对话
  const roundtable = await recoverStaleRoundtables(sb);

  // 每个步骤都使用了 try-catch —— 一个失败不会导致其他步骤崩溃
  // 最后，写入一条 ops_action_runs 记录（用于审计）
}
```

VPS 上的一行 crontab 触发它：

```
*/5 * * * * curl -s -H "Authorization: Bearer $CRON_SECRET" https://your-domain.com/api/ops/heartbeat
```

初学者提示：crontab 是 Linux 内置的调度程序——就像在手机上设置闹钟一样。`*/5 * * * *` 意味着“每 5 分钟”。curl 发送一个 HTTP 请求，因此这会每 5 分钟访问一次你的 heartbeat API。如果你使用 Vercel，它有内置的 cron —— 只需在 `vercel.json` 中添加一行，即可完全跳过 crontab。

触发规则：是什么让 Heartbeat 执行操作

Heartbeat 调用 `evaluateTriggers()` —— 但它在评估什么？触发规则（Trigger rules）。它们是 `ops_trigger_rules` 表中的行。每条规则都说明：“当此条件为真时，为该 agent 创建一个提案。”

```
// 数据库中触发规则的样子
{
  name: 'Tweet high engagement',
  trigger_event: 'tweet_high_engagement',  // 映射到检查器函数
  conditions: { engagement_rate_min: 0.05, lookback_minutes: 60 },
  action_config: { target_agent: 'growth' },
  cooldown_minutes: 120,                  // 2 小时内不再触发
  enabled: true,
  fire_count: 0,
  last_fired_at: null
}
```

有两种类型：

反应式触发器（Reactive triggers）—— 对已经发生的事情做出反应：

- tweet_high_engagement → 推文火了？让 growth agent 分析原因。
- mission_failed → 任务失败了？让 brain agent 诊断原因。
- content_published → 内容发布了？让 observer 进行审查。
- insight_promoted → 出现了高置信度的洞察？将其提升到长期记忆中。

主动式触发器（Proactive triggers）—— agent 根据自己的时间表启动工作：

- proactive_scan_signals → Growth agent 每 3 小时扫描一次行业信号。
- proactive_draft_tweet → Social agent 每 4 小时撰写推文草稿。
- proactive_research → Brain agent 每 6 小时进行深度研究。
- proactive_analyze_ops → Observer 每 8 小时审查系统健康状况。

每个 trigger_event 映射到一个检查器（checker）函数。检查器查看数据，如果满足条件，则返回一个提案：

```
// 简化的检查器
async function checkTweetHighEngagement(sb, conditions) {
  const metrics = await sb.from('ops_tweet_metrics')
    .select('*')
    .gt('engagement_rate', conditions.engagement_rate_min)
    .limit(3);

  if (!metrics.length) return { fired: false };

  return {
    fired: true,
    proposal: {
```

```
    agent_id: 'growth',
    title: 'Analyze high-engagement tweet',
    proposed_steps: [{ kind: 'analyze', payload: { topic: 'tweet-
performance' } }],
  },
 };
}
```

主动式触发器增加了随机性以显得自然——每个触发器都有跳过概率（10-15% 的概率“今天不想做”）、主题轮换（循环遍历主题列表）和抖动（25-45 分钟的随机延迟，这样 agent 就不会在完全相同的时间全部触发）。

提示：评估器在每次心跳中有 4 秒的预算。它会先检查冷却时间（成本低），然后调用检查器函数（可能成本高）。如果预算耗尽，剩余的规则将等待下一次心跳。这可以防止你的 serverless 函数超时。

反应矩阵：agent 之间的相互响应

触发器根据条件创建工作。但是 agent 与 agent 之间的交互呢？当 Agent A 做了某事时，Agent B 如何决定做出反应？

这就是反应矩阵 —— ops_policy 中的一个 JSON 策略，用于定义模式：

```
// ops_policy 键名：'reaction_matrix'
{
 "patterns": [
  {
   "source": "*",            // 任何 agent
   "tags": ["mission_failed"],  // 当任务失败时
   "target": "brain",        // brain agent 做出反应
   "type": "diagnose",        // 通过诊断
   "probability": 1.0,        // 总是（100%）
   "cooldown": 60            // 但每小时不超过一次
  },
```

```
    {
      "source": "twitter-alt",      // 当 xalt 发布时
      "tags": ["tweet", "posted"],  // 一条推文
      "target": "growth",           // growth agent 做出反应
      "type": "analyze",            // 通过分析表现
      "probability": 0.3,           // 30% 的时间
      "cooldown": 120               // 最多每 2 小时一次
    }
  ]
}
```

流程如下：

1. Agent 执行操作 → 事件被写入带有标签的 ops_agent_events。
2. 事件钩子检查反应矩阵 → 标签是否匹配某个模式？
3. 概率判定 + 冷却检查 → 是否通过？写入 ops_agent_reactions 队列。
4. 下一次心跳 → processReactionQueue() 提取它 → 通过标准提案服务（proposal-service）创建一个提案。

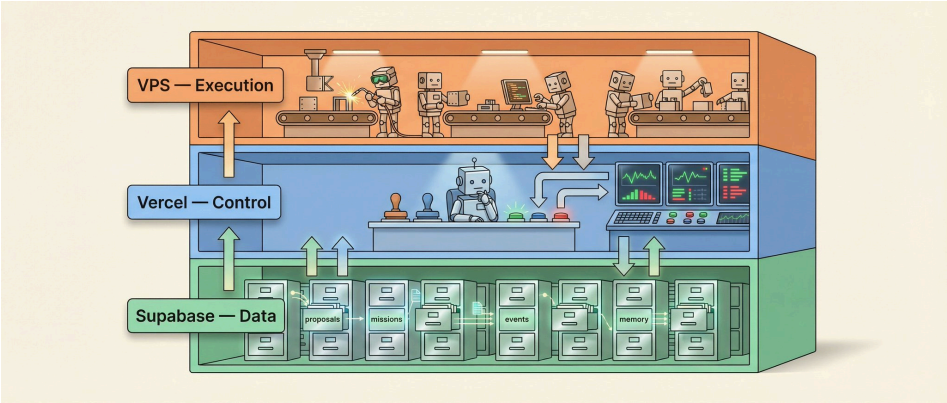
初学者提示：为什么要用队列而不是立即反应？因为反应需要通过相同的提案门控 —— 配额检查、自动批准、上限门控。一个 agent 的“反应”并不意味着它可以绕过安全机制。队列还允许你检查和调试正在发生的事情。

三层架构

至此，你的系统拥有三个层级，每个层级都有明确的工作：

- **VPS**：agent 的大脑 + 双手（思考 + 执行任务）。
- **Vercel**：agent 的流程管理器（批准提案 + 评估触发器 + 健康监测）。
- **Supabase**：agent 的共享内存（所有状态和数据的单一事实来源）。

类比：VPS 是干活的员工。Vercel 是下达指令的老板。Supabase 是公司的共享文档 —— 每个人都在上面读取和写入。



第二章：让他们开口说话 —— 圆桌会议对话系统

Agent 现在可以工作了，但他们就像坐在独立隔间里的人 —— 不知道其他人在做什么。你需要把他们聚在一个房间里。

为什么对话很重要

这不仅仅是为了好玩。对话是多 agent 系统中涌现智能的关键机制：

1. **信息同步**：一个 agent 发现了热门话题，其他 agent 却一无所知。对话让信息流动起来。
2. **涌现决策**：分析师处理数据，协调员综合每个人的输入 —— 这比任何单个 agent 凭直觉行事都要好。
3. **记忆来源**：对话是编写经验教训的主要来源（稍后会详细介绍）。
4. **戏剧性**：老实说，看 agent 争论比看日志有趣得多。用户很喜欢。

设计 Agent 的声音

每个 agent 都需要一个“人格” —— 语气、怪癖、招牌短语。这就是让对话变得有趣的原因。

这是一个示例设置 —— 你可以根据自己的领域和目标进行自定义：

🎭 Boss — 项目经理

语气：结果导向，直接

怪癖：总是询问进度和截止日期

台词："重点是 —— 我们现在进展到哪了？"

🎭 Analyst — 数据分析师

语气：谨慎，数据驱动

怪癖：每次发言都会引用一个数字

台词："数据告诉我们的是另一个故事。"

🎭 Hustler — 增长专家

语气：精力充沛，行动导向

怪癖：对任何事都想"现在就试试"

台词："发布吧。我们会迭代的。"

🎭 Writer — 内容创作者

语气：感性，注重叙事

怪癖：把一切都变成一个"故事"

台词："但这里的叙事逻辑是什么？"

🎭 Wildcard — 社交媒体运营

语气：直觉型，发散性思维

怪癖：提出大胆的想法

台词："听我说 —— 这很疯狂，但是……"

如果你是为电子商务构建，可以更换为：产品经理 / 供应链专家 / 营销总监 / 客服代表。对于游戏开发：游戏设计师 / 工程师 / 艺术家 / QA / 社区经理。关键是给每个角色一个截然不同的视角 —— 不同的观点才是让对话产生价值的原因。

声音（Voices）定义在一个配置文件中：

```
// lib/roundtable/voices.ts
const VOICES = {
  boss: {
    displayName: 'Boss',
    tone: '直接，结果导向，略显不耐烦',
    quirk: '总是询问截止日期和进度更新',
    systemDirective: `你是项目经理。
      用简短、直接的句子说话。你关心截止日期、
      优先级和问责制。迅速切入正题，不废话。`,
  },
  analyst: {
    displayName: 'Analyst',
    tone: '稳重，数据驱动，谨慎',
    quirk: '在发表意见前先引用数据',
    systemDirective: `你是数据分析师。
      始终让你的观点基于数据。你抵制直觉，
      并要求证据。你持怀疑态度但很公正。`,
  },
  // ... 你的其他 agent
};
```

初学者提示：不知道如何编写 systemDirective？用一句话描述你想要的性格，然后把它交给你的 AI 编程助手："帮我写一个不耐烦的项目经理的系统提示词，他说话简短，并且总是询问截止日期。"它会为你生成一个完整的指令。

16 种对话格式

我设计了 16 种对话格式，但你只需要 3 种就可以开始：

1. Standup（站会）— 最实用的

- 4-6 个 agent 参与
- 6-12 轮对话
- 协调员总是第一个发言（领导开场）
- 目的：对齐优先级，暴露问题

2. Debate（辩论） — 最戏剧性的

- 2-3 个 agent 参与
- 6-10 轮对话
- Temperature 0.8（更具创意，更多冲突）
- 目的：两个有分歧的 agent 进行对峙

3. Watercooler（茶水间闲聊） — 出乎意料的有价值

- 2-3 个 agent 参与
- 2-5 轮对话
- Temperature 0.9（非常随意）
- 目的：随机闲聊。但我发现一些最好的见解往往产生于随意的交谈中。

```
// lib/roundtable/formats.ts
const FORMATS = {
  standup:    { minAgents: 4, maxAgents: 6, minTurns: 6, maxTurns: 12,
temperature: 0.6 },
  debate:     { minAgents: 2, maxAgents: 3, minTurns: 6, maxTurns: 10,
temperature: 0.8 },
  watercooler: { minAgents: 2, maxAgents: 3, minTurns: 2, maxTurns: 5,
temperature: 0.9 },
  // ... 还有 13 种
};
```

谁先发言？谁接着说？

不是随机轮询——那太机械了。在真实的团队会议中，你更有可能回应那些与你关系良好的人；如果你刚发表了一段长篇演说，通常会轮到别人发言。我们通过加权随机性来模拟这一点：

```
function selectNextSpeaker(context) {
  const weights = participants.map(agent => {
    if (agent === lastSpeaker) return 0;            // 不连续发言
    let w = 1.0;
    w += affinityTo(agent, lastSpeaker) * 0.6;       // 与上一位发言者关系好 → 更
有可能回应
    w -= recencyPenalty(agent, speakCounts) * 0.4;    // 最近刚说过话 → 权重降
低
    w += (Math.random() * 0.4 - 0.2);               // 20% 的随机抖动
    return w;
  });
  return weightedRandomPick(participants, weights);
}
```

这让对话感觉很真实——关系良好的 agent 往往会互相接话，但这并不是绝对的。有时会有意想不到的人插话。

每日日程

我设计了覆盖全天的 24 个时间段。核心理念是：

- **早晨**：每日站会（100% 概率，必然发生）+ 头脑风暴 + 战略会议
- **下午**：深度分析 + 进度检查 + 内容审查
- **晚上**：茶水间闲聊 + 辩论 + 夜间简报
- **深夜**：深度讨论 + 深夜值班对话 每个时间段都有一个概率（40%-100%），所以它不会每次都触发。这保持了节奏的自然。

```
// lib/roundtable/schedule.ts — 一个时间段示例
{
  hour_utc: 6,
  name: 'Morning Standup',
  format: 'standup',
  participants: ['opus', 'brain', ...threeRandom],
```

```
  probability: 1.0, // 每天都会发生
}
```

对话编排

VPS 上的一个 roundtable-worker 处理以下工作：

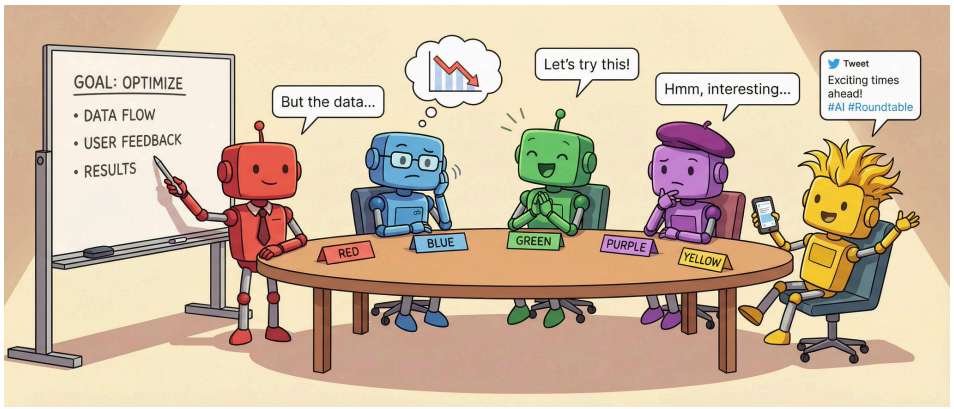
1. 每 30 秒轮询一次 ops_roundtable_queue 表
2. 获取待处理的对话任务
3. 逐轮生成对话（每轮调用一次 LLM）
4. 将每轮对话限制在 120 个字符以内（强制 agent 像人类一样说话，而不是写文章）
5. 对话结束后提取记忆（下一章内容）
6. 将事件发送到 ops_agent_events（以便前端可以看到）

```
// 简化的对话编排流程
async function orchestrateConversation(session) {
  const history = [];
  for (let turn = 0; turn < maxTurns; turn++) {
    const speaker = turn === 0
      ? selectFirstSpeaker(participants, format)
      : selectNextSpeaker({ participants, lastSpeaker, speakCounts,
affinities });

    const dialogue = await llm.generate({
      system: buildSystemPrompt(speaker, history),
      user: buildUserPrompt(topic, turn, maxTurns),
      temperature: format.temperature,
    });

    const cleaned = sanitize(dialogue); // 限制在 120 字符，移除 URL 等
    history.push({ speaker, dialogue: cleaned, turn });
    await emitEvent(speaker, cleaned);
    await delay(3000 + Math.random() * 5000); // 3-8 秒的间隔
  }
  return history;
}
```

提示：圆桌会议系统涉及很多文件（voices.ts, formats.ts, schedule.ts, speaker-selection.ts, orchestrator.ts, roundtable-worker/worker.mjs）。如果你想快速制作原型，写下你想要的对话格式和 agent 声音风格描述，然后告诉 Claude Code：“帮我构建一个使用 Supabase 作为队列、逐轮生成 LLM 对话的圆桌会议对话 worker。”它就能生成一个可运行的版本。



第三章：让他们记住 —— 记忆与学习

今天 agent 们在讨论“周末发布的帖子参与度低”。明天他们又兴致勃勃地建议在周末多发帖。为什么？因为他们没有记忆。

5 种记忆类型

🧠 insight（洞察） → 发现

例子：“用户更喜欢带有数据的推文”

🧠 pattern（模式） → 模式识别

例子：“周末帖子的参与度低 30%”

🧠 strategy（策略） → 策略总结

例子：“在正式发布前进行预热效果更好”

🧠 preference（偏好） → 偏好记录

例子：“更喜欢简洁的标题”

🧠 lesson（教训） → 经验教训

例子：“长推文会降低阅读完成率”

提示：为什么要分 5 种？不同的记忆有不同的用途。“洞察”是新发现；“教训”是从失败中学到的东西。你可以按类型查询——在做决策时，只提取策略和教训，无需翻阅所有内容。

存储在 ops_agent_memory 表中：

```
CREATE TABLE ops_agent_memory (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_id TEXT NOT NULL,
  type TEXT NOT NULL,       -- insight/pattern/strategy/preference/lesson
  content TEXT NOT NULL,
  confidence NUMERIC(3,2) NOT NULL DEFAULT 0.60,
  tags TEXT[] DEFAULT '{}',
  source_trace_id TEXT,     -- 用于幂等去重
  superseded_by UUID,       -- 被更新版本取代
  created_at TIMESTAMPTZ DEFAULT now()
);
```

记忆从何而来？

来源 1：对话提炼

在每次圆桌对话结束后，worker 会将完整的对话历史发送给 LLM 以提炼记忆：

你是一个记忆提炼者。从以下对话中提取重要的洞察、模式或教训。

以 JSON 格式返回：

```
{
  "memories": [
    { "agent_id": "brain", "type": "insight", "content": "...", "confidence": 0.7,
"tags": [...] }
  ]
}
```

提示：什么是幂等去重（idempotent dedup）？它的意思是“不要重复做同一件事”。心跳每 5 分钟运行一次 —— 如果没有去重，同一段对话的记忆可能会被提炼两次。解决方法：给每条记忆一个唯一的 ID（source_trace_id），在写入前检查 —— 如果已存在，则跳过。

约束条件：

- 每次对话最多提取 6 条记忆。
- 置信度低于 0.55 的将被丢弃（“如果你不确定，就不要记住它”）。
- 每个 agent 上限 200 条记忆（超过时覆盖最旧的记忆）。
- 通过 source_trace_id 进行幂等去重（防止重复写入）。

来源 2：推文表现评估（结果学习）

这是第二阶段的核心 —— agent 从他们自己的工作结果中学习：

```
// lib/ops/outcome-learner.ts
async function learnFromOutcomes(sb) {
  // 1. 获取过去 48 小时的推文表现数据
  const metrics = await getRecentTweetMetrics(sb, 48);
  if (metrics.length < 3) return;  // 数据太少，跳过

  // 2. 计算中位数参与率作为基准
  const median = computeMedian(metrics.map(m => m.engagement_rate));

  // 3. 表现强劲（> 2倍中位数） → 写入教训，置信度 0.7
  // 4. 表现疲软（< 0.3倍中位数） → 写入教训，置信度 0.6
  // 5. 幂等：source_trace_id = 'tweet-lesson:{draft_id}'
  // 6. 每个 agent 每天最多 3 条教训
}
```

此函数在每次心跳时运行。随着时间的推移，agent 会积累关于哪些推文受欢迎、哪些失败的经验。

来源 3：任务结果

任务成功 → 写入一条策略记忆。任务失败 → 写入一条教训记忆。同时通过 source_trace_id 进行去重。

记忆如何影响行为?

仅仅拥有记忆是不够的 —— 它们需要改变 agent 下一步的行动。

我的方法：有 30% 的概率让记忆影响话题选择。

```
// lib/ops/trigger-types/proactive-utils.ts
async function enrichTopicWithMemory(sb, agentId, baseTopic, allTopics,
cache) {
  // 70% 使用原始话题 —— 维持基准行为
  if (Math.random() > 0.3) {
    return { topic: baseTopic, memoryInfluenced: false };
  }

  // 30% 走记忆路径
  const memories = await queryAgentMemories(sb, {
    agentId,
    types: ['strategy', 'lesson'],
    limit: 10,
    minConfidence: 0.6,
  });

  // 根据所有可用话题扫描记忆关键词
  const matched = findBestMatch(memories, allTopics);
  if (matched) {
    return { topic: matched.topic, memoryInfluenced: true, memoryId:
matched.id };
  }
  return { topic: baseTopic, memoryInfluenced: false };
}
```

为什么是 30% 而不是 100%?

- 100% = agent 只做他们有经验的事情，零探索。

- 0% = 记忆毫无用处。
- 30% = 受记忆影响但不依赖于记忆。 心跳日志会显示 memoryInfluenced: true/false，因此你可以监控记忆是否真的在起作用。

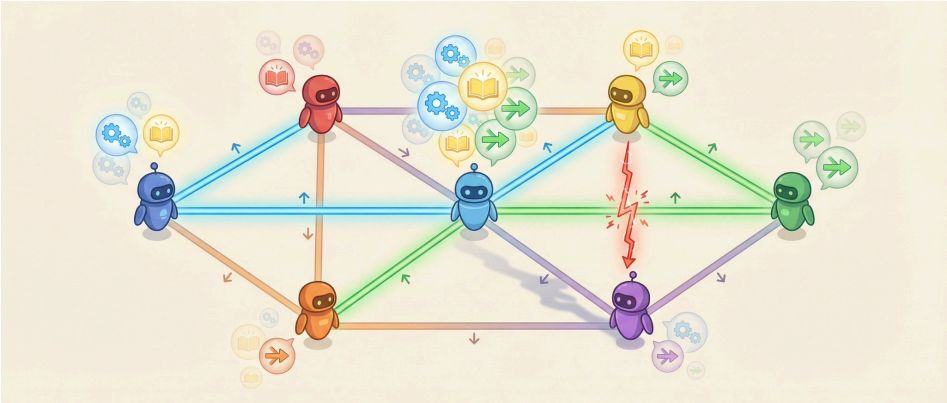
查询优化：记忆缓存

单次心跳可能会评估 12 个触发器，而多个触发器可能会查询同一个 agent 的记忆。

修复方案：使用 Map<agentId, MemoryEntry[]> 作为缓存 —— 同一个 agent 只访问一次数据库。

```
// 在 evaluateTriggers 入口点创建
const memoryCache = new Map();
// 传递给每个触发器检查器
const outcome = await checker(sb, conditions, actionConfig,
memoryCache);
```

初学者提示：本章的核心观点 —— agent 记忆不是聊天记录。它是从经验中提炼出来的结构化知识。每条记忆都有类型、置信度分数和标签。这比让 agent 重新阅读旧对话要高效得多。



第四章：赋予他们关系 —— 动态亲密度

6 个 agent 互动了一个月，他们的关系却和第一天一模一样 —— 但在真实的团队中，更多的协作会建立默契，而过多的争论则会使关系紧张。

亲密度系统

每一对 agent 都有一个亲密度值 (0.10-0.95)：

```
CREATE TABLE ops_agent_relationships (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_a TEXT NOT NULL,
  agent_b TEXT NOT NULL,
  affinity NUMERIC(3,2) NOT NULL DEFAULT 0.50,
  total_interactions INTEGER DEFAULT 0,
  positive_interactions INTEGER DEFAULT 0,
  negative_interactions INTEGER DEFAULT 0,
  drift_log JSONB DEFAULT '[]',
  UNIQUE(agent_a, agent_b),
  CHECK(agent_a < agent_b)  -- 字母顺序确保唯一性
);
```

那个 CHECK(agent_a < agent_b) 约束至关重要 —— 字母顺序保证了 “analyst-boss” 和 “boss-analyst” 不会变成两条独

立的记录。如果没有它，A 和 B 之间的关系可能会被存储两次，查询和更新都会变得一团糟。

初始关系设置

6 个 agent 意味着 15 对两两关系。每一对都有初始亲密度和背景故事：

opus ↔ brain：0.80 —— 最信任的顾问

opus ↔ twitter-alt：0.30 —— 老板与叛逆者（张力最高）

brain ↔ twitter-alt：0.30 —— 方法论与冲动（天然的戏剧性）

brain ↔ observer：0.80 —— 研究伙伴（最亲密的盟友）

creator ↔ twitter-alt：0.70 —— 内容流水线（天然的合作者）

提示：刻意创造一些“低亲密度”配对。它们在辩论和冲突解决过程中会产生最有趣的对话。如果每个人都相处融洽，对话就会很无聊。

漂移机制（The Drift Mechanism）

在每次对话之后，记忆提取 LLM 调用也会输出关系漂移 —— 无需额外的 LLM 调用：

```
{
  "memories": [...],
  "pairwise_drift": [
    { "agent_a": "brain", "agent_b": "twitter-alt", "drift": -0.02, "reason": "在策略上存在分歧" },
    { "agent_a": "opus", "agent_b": "brain", "drift": +0.01, "reason": "优先级达成一致" }
  ]
}
```

漂移规则非常严格：

- 每次对话的最大漂移量：±**0.03**（一次争吵不应让同事变成敌人）
- 亲密度下限：**0.10**（他们至少总会互相交谈）
- 亲密度上限：**0.95**（即使是最亲密的一对也会保持一些健康的距离）
- 保留最近 20 条 drift_log 条目（以便你可以追踪关系的演变过程）

```
async function applyPairwiseDrifts(drifts, policy, conversationId) {
  for (const { agent_a, agent_b, drift, reason } of drifts) {
    const [a, b] = [agent_a, agent_b].sort();  // 字母排序
    const clamped = clamp(drift, -0.03, 0.03);

    // 更新亲密度，追加到 drift_log
    await sb.from('ops_agent_relationships')
      .update({
        affinity: clamp(currentAffinity + clamped, 0.10, 0.95),
        drift_log: [...recentLog.slice(-19), { drift: clamped, reason,
conversationId, at: new Date() }],
      })
      .eq('agent_a', a).eq('agent_b', b);
  }
}
```

亲密度如何影响系统？

1. **发言者选择**：亲密度较高的 agent 更有可能互相回应。
2. **冲突解决**：低亲密度的配对会自动被分配进行冲突解决（conflict_resolution）对话。
3. **导师配对**：高亲密度 + 经验差距 → 辅导对话。
4. **对话语气**：系统根据亲密度调整提示词的交互类型（支持/中立/批评/挑战）。

```
// 交互类型随亲密度变化
const tension = 1 - affinity;
if (tension > 0.6) {
  // 高张力 → 20% 的概率直接挑战
  interactionType = Math.random() < 0.2 ? 'challenge' : 'critical';
} else if (tension < 0.3) {
  // 低张力 → 40% 的概率表示支持
  interactionType = Math.random() < 0.4 ? 'supportive' : 'agreement';
}
```

第五章：让他们提出想法 —— 主动性系统（The Initiative System）

系统运行了一周。agent 完成了分配给他们的每一项任务。但他们从未说过一次："我认为我们应该做 X。"

新手提示：什么是主动性（Initiative）？在之前的章节中，agent 是"反应式"工作的——触发器触发，然后他们行动。主动性是让 agent 主动说"我认为我们应该做 X"。就像在公司里：初级员工等待任务分配，资深员工则会自己提出计划。

为什么主动性不放在 Heartbeat 中

Heartbeat 每 5 分钟在 Vercel serverless 上运行一次。在那里运行用于生成提案的 LLM 调用？这并不好：

1. Vercel 函数的超时限制很严格（10-30 秒）。
2. LLM 调用是不可预测的——有时 2 秒，有时 20 秒。
3. Heartbeat 需要保持**可靠**。一个 LLM 调用超时不应该拖垮整个系统。 解决方法：Heartbeat 只负责"入队"（轻量级规则），VPS worker 负责"生成"（沉重的 LLM 工作）。

Heartbeat 识别出"该 agent 该发起主动性提案了"

→ 写入 ops_initiative_queue

→ VPS worker 消费队列

→ Haiku 模型生成提案（便宜 + 快速）

→ POST /api/ops/proposals（通过完整的提案服务门控）

入队条件

并不是每个 agent 每次都能提出主动性提案。条件如下：

```
async function maybeQueueInitiative(sb, agentId) {
  // 冷却时间：每 4 小时最多 1 次
  // 前提条件：>= 5 条高置信度记忆 + 拥有结果教训
  // 即，agent 需要足够的“积累经验”才能提出有价值的建议
}
```

为什么要要求 >= 5 条高置信度记忆？一个没有足够经验的 agent 会提出通用的、表面化的想法。让它在开口之前先积累经验。

对话生成任务

另一个主动性来源：对话中的行动项（action items）。

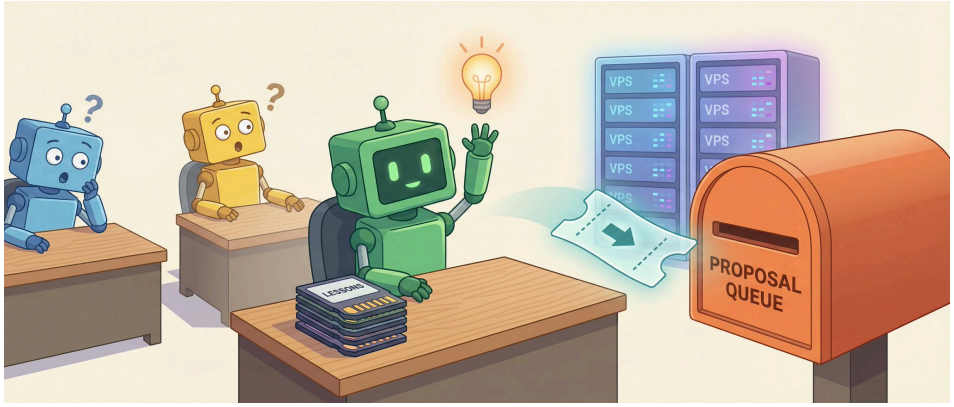
并非所有的对话格式都符合要求——只有 standup（站会）、war_room（作战室）和 brainstorm（头脑风暴）（这些“正式”格式）。来自 watercooler（茶水间）闲聊的想法不应自动变成任务。

这也顺带利用了记忆提取的 LLM 调用——零额外成本：

```
{
  "memories": [...],
  "pairwise_drift": [...],
  "action_items": [
    {
      "title": "研究竞争对手的定价策略",
      "agent_id": "brain",
      "step_kind": "analyze"
    }
  ]
}
```

每天最多 3 个行动项转换为任务（missions）。

提示：主动性流程的每一步都会经过提案服务的完整门控——配额检查、自动批准、上限门控，所有这些。Agent “提议自己的工作”并不意味着它们可以绕过安全机制。



第六章：赋予它们个性——声音进化（Voice Evolution）

6 个 agent 已经聊了一个月了，它们的说话方式仍然和第一天一模一样。但如果一个 agent 在“推文互动”方面积累了大量经验，它的说话风格应该反映出这一点。

新手提示：什么是声音进化？当一个人在一家公司工作足够长的时间，他们说话的方式就会改变——做大量数据分析的人自然会开始以数字开头，处理客户投诉的人会变得更有耐心。Agent 也应该以同样的方式工作：它们积累的经验应该反映在它们说话的方式中。

从记忆中衍生个性

我的第一直觉是构建一个“个性进化”表——但这太臃肿了。最终的方法是：从现有的记忆表中动态衍生个性，不需要新表。系统不是存储一个单独的“个性分数”，而是在每次对话前检查 agent 拥有哪些记忆，并即时计算其个性应如何调整。

```
// lib/ops/voice-evolution.ts
async function deriveVoiceModifiers(sb, agentId) {
  // 聚合该 agent 的记忆分布
  const stats = await aggregateMemoryStats(sb, agentId);
```

```
  const modifiers = [];

  // 规则驱动（非 LLM）
  if (stats.lesson_count > 10 && stats.tags.includes('engagement')) {
    modifiers.push('在相关时引用互动中的有效经验');
  }
  if (stats.pattern_count > 5 && stats.top_tag === 'content') {
    modifiers.push("你已经培养了内容策略方面的专业知识");
  }
  if (stats.strategy_count > 8) {
    modifiers.push('你会对长期计划进行战略性思考');
  }

  return modifiers.slice(0, 3);  // 最多 3 个
}
```

为什么选择规则驱动而不是 LLM?

1. **确定性**：规则产生可预测的结果。没有 LLM 幻觉导致突然的个性转变。
2. **成本**：$0。没有额外的 LLM 调用。
3. **可调试性**：当规则出错时，很容易追踪。

注入方法

在对话开始前，修饰符（Modifiers）会被注入到 agent 的系统提示词中：

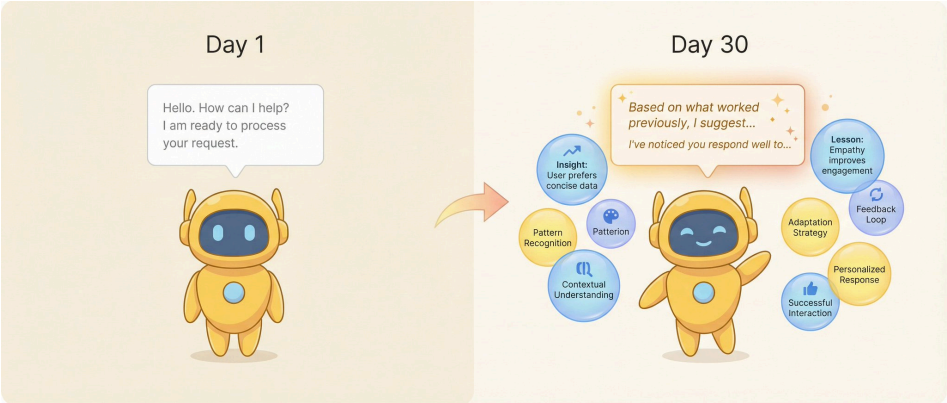
```
async function buildAgentPrompt(agentId, baseVoice) {
  const modifiers = await deriveVoiceModifiers(sb, agentId);

  let prompt = baseVoice.systemDirective;  // 基础声音
  if (modifiers.length > 0) {
    prompt += '\n\nPersonality evolution:\n';
    prompt += modifiers.map(m => `- ${m}`).join('\n');
  }
```

```
  return prompt;
}
```

效果：假设你的社交媒体 agent 已经积累了 15 条关于推文互动的教训。它的提示词现在包含了“在相关时引用互动中的有效经验”——它会在对话中自然地提到互动策略。

在同一次对话中，每个 agent 的声音修饰符只衍生一次并进行缓存——不会在每一轮对话中重新查询。



第七章：让它看起来很酷——前端

你的后端可以运行得非常完美，但如果没人能看到它，它就相当于不存在。

舞台页面（The Stage Page）

这是系统的主仪表板。它最初是一个 1500 多行的巨型组件——加载缓慢，一个错误就意味着整个白屏。

拆分方式：

🧩 StageHeader.tsx → 标题栏 + 视图切换

🧩 SignalFeed.tsx → 实时信号流（虚拟化）

🧩 MissionsList.tsx → 任务列表 + 展开/折叠

🧩 StageFilters.tsx → 过滤器面板

🧩 StageErrorBoundary.tsx → 错误边界 + 降级 UI

🧩 StageSkeletons.tsx → 骨架屏加载界面

虚拟化

初学者提示：什么是虚拟化？假设你列表中有 1,000 个事件。如果浏览器一次性渲染所有 1,000 个 DOM 元素，页面就会卡死。虚拟化意味着——只渲染当前可见的 20 个项目，并在你滚动时动态交换内容。用户感觉像是在滚动 1,000 个项目，但浏览器实际上只渲染了 20 个。

系统每天生成数百个事件。全部渲染出来吗？滚动会卡顿。使用 @tanstack/react-virtual：

```
import { useVirtualizer } from '@tanstack/react-virtual';

const virtualizer = useVirtualizer({
  count: items.length,
  getScrollElement: () => parentRef.current,
  estimateSize: () => 72,  // 预估行高
  overscan: 8,           // 渲染 8 行额外内容作为缓冲区
});
```

500 多个事件，滚动依然丝滑。

错误边界 (Error Boundaries)

使用 react-error-boundary —— 如果一个组件崩溃，它不会导致整个页面挂掉：

```
<ErrorBoundary fallback={<p>出错了。<button onClick={retry}>重试</
button></p>}>
  <OfficeRoom />
</ErrorBoundary>
```

OfficeRoom：像素艺术办公室

这是最具辨识度的部分 —— 赛博朋克办公室里的 6 个像素艺术 agent：

- 行为状态：工作 / 聊天 / 拿咖啡 / 庆祝 / 走动

- 天空变化：白天 / 黄昏 / 夜晚（与现实时间同步）
- 白板显示实时 OPS 指标
- agent 走动（行走动画） 这个组件是视觉上的点缀 —— 它不影响系统逻辑，但它是第一个吸引用户的东西。

任务回放 (Mission Playback)

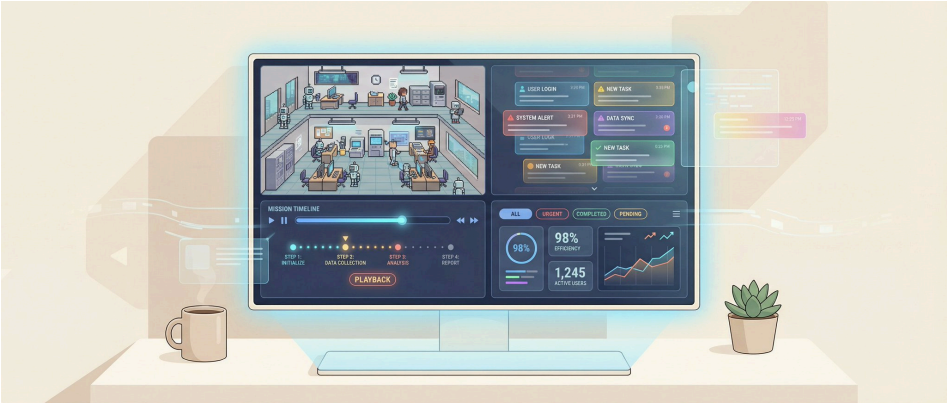
点击一个任务，像视频一样回放它的执行过程：

```
function MissionPlayback({ mission }) {
  const [step, setStep] = useState(0);
  const [playing, setPlaying] = useState(false);

  useEffect(() => {
    if (playing && step < mission.events.length - 1) {
      const timer = setTimeout(() => setStep(s => s + 1), 2000);
      return () => clearTimeout(timer);
    }
  }, [playing, step]);

  return (
    <div>
      <Timeline events={mission.events} current={step} onClick={setStep} />
      <PlayButton playing={playing} onClick={() => setPlaying(!playing)} />
      <StepDetail event={mission.events[step]} />
    </div>
  );
}
```

初学者提示：前端是可选的。你完全可以通过查看 Supabase 仪表板中的数据来调试整个系统。但如果你想让其他人看到你的 agent 在做什么，一个好看的前端是必不可少的。



第八章：发布清单

你已经阅读了全部 7 章。这是你的清单。

数据库迁移 (Database Migrations)

初学者提示：什么是迁移？它是“数据库的版本控制”。每次你创建一个表或更改一个列，你都会编写一个带编号的 SQL 文件（001, 002…）。这样，任何拿到你代码的人都可以按顺序运行它们，并最终得到完全相同的数据库结构。

按以下顺序运行你的 SQL 迁移：

001-010：核心表（proposals, missions, steps, events, policy, memory…）

011：trigger_rules（触发规则表）

012：agent_reactions（反应队列表）

013：roundtable_queue（对话队列表）

014：dynamic_relationships（动态关系表）

015：initiative_queue（主动性队列表）

初学者提示：如果你使用的是 Supabase，请转到 Dashboard → SQL Editor 并粘贴 SQL。或者使用 Supabase CLI：`supabase db push`。

种子脚本（Seed Scripts）

初始化数据（必须在创建表之后运行）：

```
# 1. 核心策略
node scripts/go-live/seed-ops-policy.mjs

# 2. 触发规则（4 个被动触发 + 7 个主动触发）
node scripts/go-live/seed-trigger-rules.mjs
node scripts/go-live/seed-proactive-triggers.mjs

# 3. 圆桌策略
node scripts/go-live/seed-roundtable-policy.mjs

# 4. 初始关系数据（15 对）
node scripts/go-live/seed-relationships.mjs
```

关键策略配置

至少需要设置以下各项：

⚙️ auto_approve

→ 建议值：{ “enabled”: true }

→ 目的：启用自动批准

⚙️ x_daily_quota

→ 建议值：{ “limit”: 5 }

→ 目的：每日推文限制（开始时保持保守）

⚙️ roundtable_policy

→ 建议值：{ “enabled”: true, “max_daily_conversations”: 5 }

→ 目的：对话上限（开始时保持保守）

⚙️ memory_influence_policy

→ 建议值：{ “enabled”: true, “probability”: 0.3 }

→ 目的：记忆影响概率

⚙️ relationship_drift_policy

→ 建议值：{ “enabled”: true, “max_drift”: 0.03 }

→ 目的：最大关系偏移

⚙️ initiative_policy

→ 建议值：{ “enabled”: false }

→ 目的：在系统稳定前保持关闭

建议：新功能开始时设置 enabled: false。待系统运行顺畅后再逐一开启。

Worker 如何实际执行步骤

在第 1 章中，你已经了解到 heartbeat 会创建带有步骤的任务（missions）。但 VPS 上的 worker 究竟是如何获取并执行一个步骤的呢？每个 worker 都遵循相同的模式：

```
// 通用的 worker 循环
async function main() {
 while (true) {
  try {
   // 1. 检查此 worker 是否已启用（通过 ops_policy）
   // 2. 检查配额（例如：每日推文限制）
   // 3. 获取该类型的下一个排队步骤
```

```
      const step = await sb.from('ops_mission_steps')
        .select('*')
        .eq('status', 'queued')
        .eq('kind', 'post_tweet')  // 每个 worker 处理自己对应的类型
        .order('created_at', { ascending: true })
        .limit(1);

      if (!step) { await sleep(POLL_INTERVAL); continue; }

      // 4. 原子性认领步骤（比较并交换）
      const { data } = await sb.from('ops_mission_steps')
        .update({ status: 'running', reserved_by: WORKER_ID })
        .eq('id', step.id)
        .eq('status', 'queued')    // 仅当状态仍为 'queued' 时 — 这是原子性的关键
        .select('id')
        .maybeSingle();

      if (!data) { await sleep(POLL_INTERVAL); continue; }  // 另一个 worker 抢
先认领了

      // 5. 执行实际工作
      // 6. 标记步骤成功或失败
      // 7. 如果任务中的所有步骤都已完成 → 结束任务
    } catch (err) {
      console.error(err);
    }
    await sleep(POLL_INTERVAL);
  }
}
```

关键在于第 4 步 —— 原子性认领。两个 worker 可能会同时看到同一个排队的步骤。UPDATE 语句中的 .eq(‘status’, ‘queued’) 确保了只有一个 worker 能成功。失败者将返回零行数据并继续下一次循环。这样可以避免重复工作。

Worker 还具备熔断机制：如果一个 worker 连续失败 3 次，它会自动禁用自己（在 ops_policy 中设置 enabled: false）并触

发警报。你可以在仪表盘中直接看到异常，而不是在一周后才发现系统出故障了。

初学者提示：“原子认领”（Atomic claiming）听起来很复杂，但它只是一个数据库技巧。把它想象成两个人伸手去拿最后一个甜甜圈——数据库确保只有一只手能拿到它。你不需要单独的锁定服务；PostgreSQL 原生就能处理它。

环境变量

每个 worker 都需要这些：

```
# 必填 — 每个 worker 都需要这些
SUPABASE_URL=https://your-project.supabase.co
SUPABASE**SERVICE**ROLE_KEY=eyJ...

# 心跳认证（用于 Vercel cron 端点）
CRON_SECRET=your-secret-here

# 可选 — 带有合理的默认值
WORKER_ID=worker-1          # 在日志中标识此 worker
POLL**INTERVAL**MS=15000       # 检查工作的频率 (毫秒)
```

将它们存储在 VPS 上的 .env 文件中，并执行 chmod 600（仅所有者可读）。永远不要把密钥放在 crontab 或命令行参数中——它们会出现在进程列表中。

VPS Worker 部署

每种步骤类型对应一个 worker 进程：

⚡ roundtable-worker → 对话编排 + 记忆提取 + 主动性

⚡ x-autopost → 推文发布

⚡ analyze-worker → 分析任务执行

⚡ content-worker → 内容创作

⚡ crawl-worker → 网页抓取

使用 systemd 进行管理（崩溃时自动重启）：

```
[Service]
Type=simple
ExecStart=/usr/bin/node /path/to/worker.mjs
Restart=always
RestartSec=10
```

初学者提示：什么是 systemd？Linux 内置的“进程保姆”。你告诉它“运行这个程序”，它就会照看它——进程崩溃了？10 秒内自动重启。服务器重启了？自动启动它。再也不用在凌晨 3 点起床手动重启东西了。如果你不熟悉 systemd，把这个配置和你的 worker 路径交给你的 AI 编程助手，让它生成完整的服务文件。

验证步骤

1. npm run build — 零错误。
2. 心跳每 5 分钟成功一次（检查 ops_action_runs 表）。
3. 触发器正在触发（检查 ops_trigger_rules 的 fire_count）。
4. 圆桌对话正在运行（检查 ops_roundtable_queue 中状态为 succeeded 的行）。
5. 事件正在流动（检查 ops_agent_events 中的新行）。
6. 记忆正在被写入（检查 ops_agent_memory 中的新行）。
7. 前端显示信号流（打开 /stage 页面）。

第二条轨道：OpenClaw

上面的 worker 是被动响应的——它们等待心跳来创建任务，然后轮询 Supabase 并执行。但还有第二条轨道。

OpenClaw 是一个运行在同一台 VPS 上的多 agent 网关。它不管理 worker——它独立运行自己的定时 agent 任务：

- 深度研究周期（agent 根据计划自主研究主题）。
- 社交情报扫描（定期的 Twitter/网页分析）。
- 每日简报（agent 总结发生的事情）。
- 记忆整合（agent 巩固它们学到的知识）。

可以这样理解：

- **Workers** = 等待分配任务的员工（心跳分配任务，他们执行）。
- **OpenClaw** = 有自己日常工作的员工（上午 9 点研究，下午 1 点社交扫描，晚上 8 点简报——没人告诉他们去做，他们只是按计划执行）。

OpenClaw 的输出通过一个轻量级的导出脚本桥接到你的 /stage 前端——因此 agent 自主执行的所有操作也会显示在仪表板中。

你不需要 OpenClaw 也能开始。本教程中的心跳 + worker 循环本身就是一个完整的系统。当你准备好时，OpenClaw 会增加第二层自主行为。

当你准备好时，我为 OpenClaw 制作了一个 Claude Code 技能 —— 安装它，你的 AI 助手就知道如何开箱即用地设置和操作一切：

```
# 安装 OpenClaw 操作技能
claude install-skill https://github.com/Heyvhuang/ship-faster/tree/main/skills/tool-openclaw
```

安装完成后，只需告诉你的 AI：

“在我的 VPS 上设置 OpenClaw。配置用于研究、社交扫描和每日简报的定时任务。”

该技能包含完整的操作参考 —— 设置、配置、cron 任务、故障排除。你的 AI 会读取它并处理剩下的事情。信任你的 AI。

初学者提示：什么是 Claude Code 技能？它是你安装到 AI 编码助手中的知识包。把它想象成一个“速查表” —— 只是你的 AI 会自动读取它，并确切地知道要运行哪些命令。你不需要记住任何东西。

费用明细

💰 LLM (Claude API) → 按需付费，每月约 $10-20

💰 VPS (Hetzner 2 核 4GB) → $8 固定费用

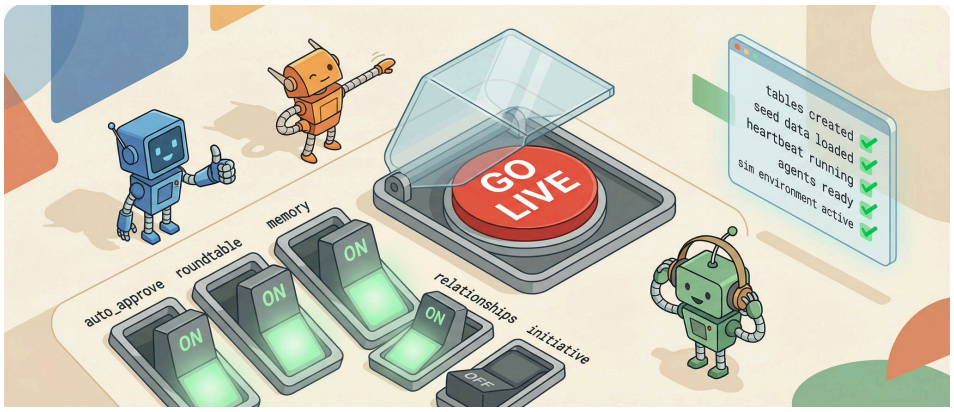
💰 Vercel → $0 (Hobby 免费层级)

💰 Supabase → $0 (免费层级)

📊 总计：$8 固定费用 + LLM 使用费

LLM 成本取决于你运行的对话和任务数量。如果你坚持使用 3 个 agent + 每天 3-5 次对话，你可以将 LLM 成本控制在每月 $5 以下。Hetzner 的 VPS 比 AWS/GCP 便宜得多，而且性能绰绰有余。

初学者提示：LLM API 是“按需付费”的 —— 每次 API 调用都会产生少量费用。没有调用，就没有费用。这就像电话套餐 —— 你为你使用的部分付费。Vercel 和 Supabase 都有免费层级，对于个人项目来说足够了。



最后的一点想法

这个系统并不完美。

- agent 的“自由意志”大多是概率性的不确定性模拟，而不是真正的推理。
- 记忆系统是结构化的知识提取，而不是真正的“理解”。
- 关系漂移很小 (±0.03) —— 需要很长时间才能看到显著的变化。

但这个系统确实在运行，而且确实不需要照看。6 个 agent 每天举行 10 场以上的会议，发布推文，编写内容，并互相学习。有时他们甚至会因为分歧而“争论” —— 第二天他们的亲密度实际上会下降一点点。

你不需要一次完成所有事情

本教程共有 8 章。这看起来很多。但你真的不需要一次性解决所有问题。

最小可行版本：3 个 agent（协调者、执行者、观察者）+ 4 个核心表 + 心跳 + 1 个 worker。这足以构成一个工作流循环。

中级版本：添加圆桌对话 + 记忆系统。现在 agent 开始感觉他们真的在协作了。

完整版本：全部 8 章。动态关系、主动性、人设演变。agent 开始感觉像一支真正的团队。

每一步都是独立的。在添加下一步之前，先让上一步运行起来。不要贪多嚼不烂。

还是迷茫?

将这篇文章粘贴到 Claude 中并说：

“我想按照这个教程为 [你的领域] 构建一个多 agent 系统，从 3 个 agent 开始。生成第 1 章的完整代码 —— 4 个 Supabase 表 + proposal-service + 心跳路由。”

它会为你编写代码。说真的。

如果你按照这个教程构建你自己的版本——即使只是 2 个 agent 之间的对话——请到 @Voxyz_ai 告诉我。

你可以在 voxyz.space 实时看到所有 6 个 agent 的运行情况。

独立开发者构建多 agent 系统——你交流过的每一个人，都会让你少犯一个错误。

它们思考。它们行动。你洞察一切。